RAGOps — Repository Structure & Module Boundaries

1. Repository Structure

```
ragops/
│
├── README.md
├── LICENSE
├── Makefile
├── pyproject.toml
├── uv.lock
├── .env.example
├── .gitignore
│
├── configs/
│   ├── base.yaml
│   ├── development.yaml
│   ├── staging.yaml
│   └── production.yaml
│
├── src/
│   └── ragops/
│       ├── api/
│       │   ├── app.py
│       │   ├── dependencies.py
│       │   ├── middleware.py
│       │   └── routes/
│       │       ├── health.py
│       │       ├── query.py
│       │       └── feedback.py
│       │
│       ├── ingestion/
│       │   ├── loaders/
│       │   │   ├── pdf.py
│       │   │   ├── html.py
│       │   │   └── markdown.py
│       │   ├── cleaners.py
│       │   ├── chunkers.py
│       │   ├── pipeline.py
│       │   └── schemas.py
│       │
│       ├── retrieval/
│       │   ├── embeddings.py
```

```
│       │   ├── vector_store.py
│       │   ├── retriever.py
│       │   ├── reranker.py
│       │   └── schemas.py
│       │
│       ├── generation/
│       │   ├── llm.py
│       │   ├── prompts.py
│       │   ├── response.py
│       │   └── schemas.py
│       │
│       ├── rag/
│       │   ├── pipeline.py
│       │   ├── query_transform.py
│       │   ├── context_builder.py
│       │   └── schemas.py
│       │
│       ├── evaluation/
│       │   ├── datasets.py
│       │   ├── retrieval.py
│       │   ├── generation.py
│       │   ├── judges.py
│       │   ├── metrics.py
│       │   ├── runner.py
│       │   └── schemas.py
│       │
│       ├── observability/
│       │   ├── tracing.py
│       │   ├── metrics.py
│       │   ├── logging.py
│       │   └── events.py
│       │
│       ├── feedback/
│       │   ├── service.py
│       │   ├── repository.py
│       │   └── schemas.py
│       │
│       ├── improvement/
│       │   ├── quality_gate.py
│       │   ├── experiment.py
│       │   ├── comparison.py
│       │   └── promotion.py
│       │
│       ├── storage/
│       │   ├── database.py
│       │   ├── models.py
│       │   └── repositories.py
│       │
```

```
│       ├── core/
│       │   ├── config.py
│       │   ├── exceptions.py
│       │   ├── logging.py
│       │   └── types.py
│       │
│       └── cli/
│           ├── ingest.py
│           ├── evaluate.py
│           └── serve.py
│
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
│
├── evaluation/
│   ├── datasets/
│   │   ├── golden.jsonl
│   │   └── regression.jsonl
│   └── reports/
│
├── data/
│   ├── raw/
│   ├── processed/
│   └── README.md
│
├── scripts/
│   ├── ingest_documents.py
│   ├── run_evaluation.py
│   └── seed_database.py
│
├── docker/
│   ├── Dockerfile.api
│   ├── Dockerfile.worker
│   └── Dockerfile.evaluation
│
├── compose/
│   └── docker-compose.yml
│
├── deploy/
│   ├── kubernetes/
│   ├── helm/
│   └── terraform/
│
├── monitoring/
│   ├── prometheus/
│   └── grafana/
```

```
│
└── .github/
    └── workflows/
        ├── ci.yml
        ├── evaluation.yml
        └── deploy.yml
```

2. Module Responsibilities

`api/`

Owns HTTP concerns:

- FastAPI application
- request validation
- authentication
- routing
- serialization
- API error handling

The API should remain thin.

```
HTTP Request
     ↓
API
     ↓
RAG Service
```

It should not directly call the vector database or LLM.

`ingestion/`

Owns the document ingestion lifecycle:

```
Document
   ↓
Loader
   ↓
Cleaner
   ↓
Chunker
```

```
    ↓
DocumentChunk[]
```

Responsible for:

- PDF/HTML/Markdown loading
- text extraction
- cleaning
- chunking
- metadata creation
- document versioning

It does not own retrieval or LLM generation.

`retrieval/`

Owns finding relevant context.

```
Query
  ↓
Embedding
  ↓
Vector Search
  ↓
Reranking
  ↓
RetrievedContext
```

Responsible for:

- embedding generation
- vector database interaction
- semantic/hybrid retrieval
- reranking

The implementation should allow replacing Qdrant with pgvector without rewriting the RAG pipeline.

`generation/`

Owns answer generation.

```
Query + Context
       ↓
```

```
Prompt
       ↓
LLM
       ↓
GeneratedResponse
```

Responsible for:

- prompt templates
- LLM clients
- generation parameters
- response parsing
- citation formatting

It should not know how documents are retrieved.

`rag/`

Owns orchestration.

```
Query
 ↓
Transform
 ↓
Retrieve
 ↓
Rerank
 ↓
Build Context
 ↓
Generate
 ↓
Response
```

This is the application-level layer connecting retrieval and generation.

`evaluation/`

Owns RAG quality measurement.

```
Golden Dataset
      ↓
RAG Pipeline
```

```
      ↓
Metrics
      ↓
EvaluationResult
```

Measures:

- Recall@K
- Precision@K
- MRR
- context relevance
- faithfulness
- answer relevance
- citation accuracy
- latency
- token usage
- cost

The evaluation system must reuse the same RAG pipeline used in production.

`observability/`

Owns system telemetry.

Tracks:

- distributed traces
- latency
- errors
- retrieval metrics
- LLM token usage
- cost
- model/version
- request metadata

Example:

```
Request
 ├── Query processing
 ├── Embedding
 ├── Retrieval
 ├── Reranking
 ├── LLM generation
 └── Response
```

`feedback/`

Owns user feedback.

```
Response
   ↓
👍 / 👎
   ↓
Feedback Store
```

Feedback can later become part of the evaluation dataset.

`improvement/`

Owns quality decisions.

```
Candidate Version
       ↓
Evaluation
       ↓
Compare With Production
       ↓
Quality Gate
       ↓
Promotion Decision
```

It determines whether a new RAG configuration should be promoted.

`storage/`

Owns persistence.

```
Application
    ↓
Repository
    ↓
Database
```

Application modules should not contain raw SQL throughout their code.

`core/`

Contains genuinely shared infrastructure:

- configuration
- common types
- exceptions
- logging

Keep this module small and avoid turning it into a miscellaneous utility folder.

---

3. Dependency Rules

The intended dependency direction is:

```
API
 ↓
RAG
 ↓
Retrieval / Generation
 ↓
Infrastructure
```

Evaluation:

```
Evaluation
 ↓
RAG
 ↓
Retrieval / Generation
```

Avoid:

```
❌ Retrieval → API
❌ Generation → Evaluation
❌ Vector Store → RAG
❌ Evaluation → API
```

Higher-level modules orchestrate lower-level capabilities.

---

4. Interface-Based Design

Important components should expose interfaces.

Example:

```
class Retriever(Protocol):

    def retrieve(
        self,
        query: str,
        top_k: int
    ) -> list[RetrievedChunk]:
        ...
```

Implementations can then be:

```
Retriever
   ├── QdrantRetriever
   ├── PgVectorRetriever
   └── HybridRetriever
```

Likewise:

```
LLMClient
   ├── OpenAIClient
   ├── AnthropicClient
   └── LocalModelClient
```

This prevents vendor-specific implementations from leaking throughout the application.

---

5. Testing Boundaries

```
tests/
├── unit/
│   ├── ingestion/
│   ├── retrieval/
│   ├── generation/
│   └── evaluation/
│
├── integration/
```

```
│   ├── test_ingestion_pipeline.py
│   ├── test_vector_store.py
│   └── test_rag_pipeline.py
│
└── e2e/
    └── test_api.py
```

Unit tests

Test individual modules in isolation.

Integration tests

Test interactions between components.

E2E tests

Test the complete user request:

```
HTTP
 ↓
API
 ↓
RAG
 ↓
Retrieval
 ↓
Generation
 ↓
Response
```

---

6. Infrastructure Separation

Application code:

```
src/
```

Infrastructure:

```
docker/
compose/
deploy/
```

```
monitoring/
.github/
```

Do not mix Kubernetes manifests, Terraform and Docker configuration into Python application modules.

---

7. Five-Layer Mental Model

```
┌──────────────────────────────────────────┐
│              API / CLI                   │
├──────────────────────────────────────────┤
│          RAG Application                  │
├──────────────────────────────────────────┤
│ Retrieval │ Generation │ Evaluation      │
├──────────────────────────────────────────┤
│      Storage / External Services         │
├──────────────────────────────────────────┤
│   Docker / K8s / Cloud / CI/CD           │
└──────────────────────────────────────────┘
```

Observability cuts across all layers:

```
API ──────────────┐
RAG ──────────────┤
Retrieval ────────┼──→ Observability
Generation ───────┤
Evaluation ───────┘
```

8. Implementation Order

Build the architecture progressively:

```
1. ingestion
      ↓
2. retrieval
      ↓
3. generation
      ↓
4. RAG orchestration
      ↓
5. evaluation
      ↓
6. observability
```

```
      ↓
7. feedback
      ↓
8. quality gates
      ↓
9. CI/CD
      ↓
10. production deployment
```

Do not create every folder as an empty placeholder. Each module should be introduced when its responsibility becomes necessary.